

CG作业3报告

李惟乐 PB23050929

内容

- 算法原理
- 接口设计
- 结果

算法原理

图像融合算法把图像看作 $\mathbb{R}^2 \rightarrow \mathbb{R}^3$ 的映射) 求解的是这样一个问题:

Input: 背景/目标图像 f^* , 源图像 g , 背景图像中的待修改区域 Ω ;

Output: 新图像 f , 插值图像 f^* 在 Ω 以外的部分, 满足 $f|_{\partial\Omega} = f^*|_{\partial\Omega}$. IDW算法插值

论文给出了 Equation.(3)

$$\min_f \iint_{\Omega} |\nabla f - \mathbf{v}|^2 \text{ with } f|_{\partial\Omega} = f^*|_{\partial\Omega}, \quad (3)$$

其选择用一个向量场 $\mathbf{v}$ 来引导区域内部颜色的变化, 如果取这个向量场为源图像的梯度场 ∇g , 就可以做到插值出来的区域具有源图像的选中区域的细节。

Equation.(3) 是一个变分问题, 可以转化为求解区域 Ω 上 Dirichlet 边界条件的 Poisson 方程 (Equation.(4)) 。

$$\Delta f = \text{div } \mathbf{v} \text{ over } \Omega, \text{ with } f|_{\partial\Omega} = f^*|_{\partial\Omega}, \quad (4)$$

论文中的 Equation.(7) 提供了要求解的线性方程组:

$$|N_p|f_p - \sum_{q \in N_p \cap \Omega} f_q = \sum_{q \in N_p \cap \partial\Omega} f_q^* + \sum_{q \in N_p} v_{pq}, \quad \text{for all } p \in \Omega, \quad (7)$$

方程组可以写成 $\mathbf{A}\mathbf{f} = \mathbf{B}$ 的格式。假设求解的区域 Ω (内部) 是一个 $W \times H$ 的矩形, 其 $W \times H$ 个像素的颜色就是待求解的变量。不妨只考虑 r 通道 (其他通道同理), 将它们“拉长”为一个 $W \times H$ 维向量 $\mathbf{r}$ 。根据 Equation.(7) 填写系数矩阵 $\mathbf{A}$ 和 $\mathbf{B}$ 的元素。

Importing gradients

对于Importing cloning, Equation.(11)给出了右侧向量 $\mathbf{B}$ 的构造方法:

$$\text{for all } (p, q), \quad v_{pq} = g_p - g_q, \quad (6)$$

因此利用三元组进行以下构造:

```
void SeamlessClone::build_matrix()
{
    ...
    SparseMatrix<double> Matrix_A(W * H, W * H);
    vector<Eigen::Triplet<double>> triplet_list;
    ...
    for (int y = 0; y < H; ++y){
        for (int x = 0; x < W; ++x){
            int idx = y * W + x;
            triplet_list.push_back(Triplet<double>(idx, idx, 4.0));
            if (x > 0) { // left
                triplet_list.push_back(Triplet<double>(idx, y * W + (x-1), -1.0));
            }
            if (x < W-1) { // right
                triplet_list.push_back(Triplet<double>(idx, y * W + (x+1), -1.0));
            }
            if (y > 0) { // up
                triplet_list.push_back(Triplet<double>(idx, (y-1) * W + x, -1.0));
            }
            if (y < H-1) { // down
                triplet_list.push_back(Triplet<double>(idx, (y+1) * W + x, -1.0));
            }
        }
    }
    Matrix_A.setFromTriplets(triplet_list.begin(), triplet_list.end());
    solver.compute(Matrix_A);
    ...
}
```

```
}
```

通过指定每一行的非零元素的位置，构造出所需方程的矩阵 A 部分。

以下为右侧向量 B 的构造示意：

```
void SeamlessClone::build_B()
{
    ...
    VectorXd B(W * H);
    for (int y = 0; y < H; ++y)
    {
        for (int x = 0; x < W; ++x)
        {
            int idx = y * W + x;
            B(idx) = 4.0 * get_source_pixel(x, y)[channel]
                - get_source_pixel(x, y - 1)[channel]
                - get_source_pixel(x, y + 1)[channel]
                - get_source_pixel(x - 1, y)[channel]
                - get_source_pixel(x + 1, y)[channel];

            // meet edge of mask
            if (x == 0){ // left edge
                B(idx) +=
                    get_target_pixel(x - 1 + Target_offset_x(), y + Target_offset_y());
            }
            if (y == 0) { // up edge
                B(idx) +=
                    get_target_pixel(x + Target_offset_x(), y - 1 + Target_offset_y());
            }
            if (x == W-1) { // right edge
                B(idx) +=
                    get_target_pixel(x + 1 + Target_offset_x(), y + Target_offset_y());
            }
            if (y == H-1) { // down edge
                B(idx) +=
                    get_target_pixel(x + Target_offset_x(), y + 1 + Target_offset_y());
            }
        }
    }
}
```

Mixing gradients

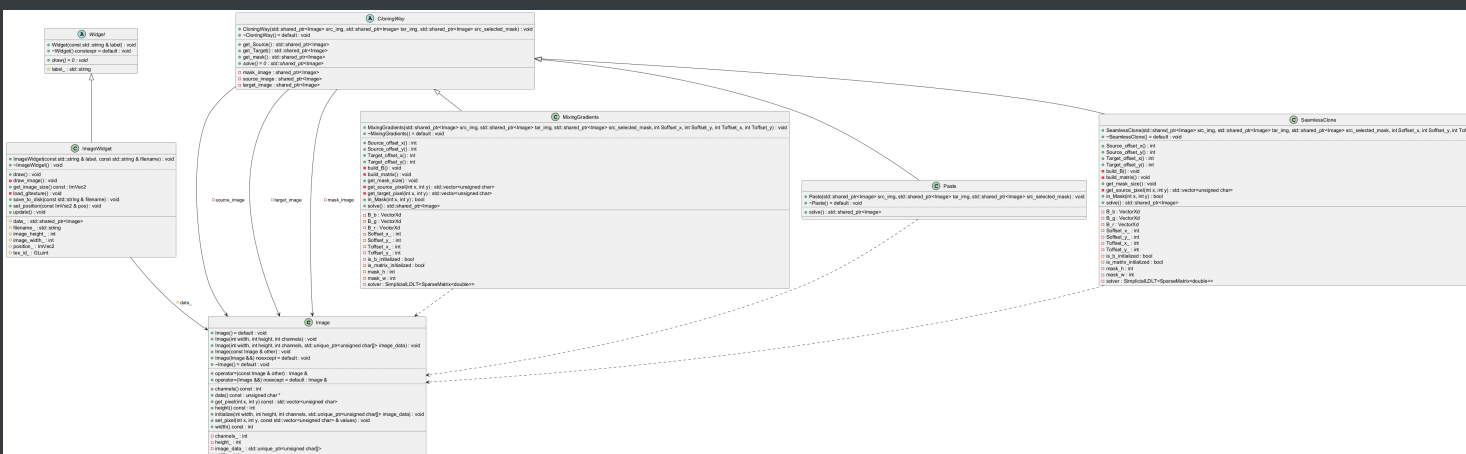
前一种方法取向量场 v 为源图像的梯度信息。混合梯度法则选用论文中 Equation.(12) 的形式

$$\text{for all } x \in \Omega, \quad v(x) = \begin{cases} |\nabla f^*(x)| & \text{if } |\nabla f^*(x)| > |\nabla g(x)|, \\ |\nabla g(x)| & \text{otherwise,} \end{cases} \quad (12)$$

和前面实现的方法唯一的不同就是右边系数 B 有所不同。具体构造示意如下：

即分别计算源图像和背景图像的梯度，哪个较大就选取哪个、

接口设计



简而言之，将clone算法抽象为了一个基类，由三个子类，即Paste，SeamlessClone和MixingGradients构成。达成了clone这一算法的不同实现。

Clone

A CloningWay	
● CloningWay(std::shared_ptr<Image> src_img, std::shared_ptr<Image> tar_img, std::shared_ptr<Image> src_selected_mask) : void ● ~CloningWay() = default : void	
● get_Source() : std::shared_ptr<Image> ● get_Target() : std::shared_ptr<Image> ● get_mask() : std::shared_ptr<Image> ● solve() = 0 : std::shared_ptr<Image>	
□ mask_image : shared_ptr<Image> □ source_image : shared_ptr<Image> □ target_image : shared_ptr<Image>	

Clone类中封装了源图像、背景图像和掩码图像，可用函数访问。同时构造了一个solve()的纯虚函数，要求子类实现。

Paste

C Paste	
● Paste(std::shared_ptr<Image> src_img, std::shared_ptr<Image> tar_img, std::shared_ptr<Image> src_selected_mask) : void ● ~Paste() = default : void	
● solve() : std::shared_ptr<Image>	

构成较为简单，solve()直接返回源图像对应区域即可。

SeamlessClone

C SeamlessClone	
● SeamlessClone(std::shared_ptr<Image> src_img, std::shared_ptr<Image> tar_img, std::shared_ptr<Image> src_selected_mask, int Soffset_x, int Soffset_y, int To ● ~SeamlessClone() = default : void	
● Source_offset_x() : int ● Source_offset_y() : int ● Target_offset_x() : int ● Target_offset_y() : int ■ build_B() : void ■ build_matrix() : void ● get_mask_size() : void ■ get_source_pixel(int x, int y) : std::vector<unsigned char> ● in_Mask(int x, int y) : bool ● solve() : std::shared_ptr<Image>	
□ B_b : VectorXd □ B_g : VectorXd □ B_r : VectorXd □ Soffset_x_ : int □ Soffset_y_ : int □ Toffset_x_ : int □ Toffset_y_ : int □ is_b_initialized : bool □ is_matrix_initialized : bool □ mask_h : int □ mask_w : int □ solver : SimplicialLDLT<SparseMatrix<double>>	

为了达成实时编辑的目标，将矩阵 A 和向量 B 进行了初始化，存在类中，并用flag进行标记。一旦初始化成功，则后续只需要直接求解方程组即可。注意这里是直接存储的求解器。

MixingGradients

C

MixingGradients

```
● MixingGradients(std::shared_ptr<Image> src_img, std::shared_ptr<Image> tar_img, std::shared_ptr<Image> src_selected_mask, int Soffset_x, int Soffset_y, int Toffset_x, int Toffset_y) : void
● ~MixingGradients() = default : void

● Source_offset_x() : int
● Source_offset_y() : int
● Target_offset_x() : int
● Target_offset_y() : int
■ build_B() : void
■ build_matrix() : void
● get_mask_size() : void
■ get_source_pixel(int x, int y) : std::vector<unsigned char>
■ get_target_pixel(int x, int y) : std::vector<unsigned char>
● in_Mask(int x, int y) : bool
● solve() : std::shared_ptr<Image>

□ B_b : VectorXd
□ B_g : VectorXd
□ B_r : VectorXd
□ Soffset_x_ : int
□ Soffset_y_ : int
□ Toffset_x_ : int
□ Toffset_y_ : int
□ is_b_initialized : bool
□ is_matrix_initialized : bool
□ mask_h : int
□ mask_w : int
□ solver : SimplicialLDLT<SparseMatrix<double>>
```

接口与SeamlessClone基本保持一致，主要区别在于build_B()的实现。

结果

按 Paste -> SeamlessClone -> Mixing Gradients 的顺序



后续扩展

在编写程序过程中，出现过以下问题：

- 边界控制问题。图像处理过程中有源图像、选中区域和背景图像三个对象，其中的边界控制容易出现问题

目前该程序还可以进行如下改进：

- 增加扫描线算法。目前已经完成了椭圆和自由绘制的接口创建，后续可以添加。